JavaScript Lab — Practical 11 (VA 11)

Course: JavaScript Programming Laboratory (Semester V)
Practical No.: 11 (VA 11)
Aim: Use JavaScript timers (setTimeout(), setInterval()) and ES6 modules (import/export) to build a dynamic countdown or class scheduler with real-time updates and responsive layout.
Student Name: Samruddhi Kalbande
PRN: 24070521278
Institution: Symbiosis Institute of Technology (SIT), Nagpur

📌 Aim & Objectives

1. Implement JavaScript timing mechanisms (`setInterval()` , `setTimeout()`) to power real-time clocks and dynamic countdown trackers.
2. Structure modular JavaScript applications using **ES6 Modules** (`export` , `import` , `type="module"`).
3. Build a production-grade **Class Scheduler** with live status determination ("In progress", "Starts in", "Completed") and schedule collision detection.
4. Design responsive, accessible UI layouts incorporating ARIA live regions (`aria-live="polite"`).
5. Implement specialized case studies demonstrating form event lifecycle (`focusin` , `focusout` , `change` , `submit`), `FormData` serialization, Web Storage caching, and CSS animation controllers.

📂 Project Structure

```
pract 11/
├── README.md                    # Comprehensive Practical 11 documentation
├── README.pdf                   # PDF export of laboratory documentation
├── PRACTICAL/                   # Class Scheduler Web Application
│   ├── index.html               # Accessible scheduler UI with clock & schedule cards
│   ├── app.js                   # Main application logic, DOM binding & interval loops
│   ├── scheduler.js             # ES6 module containing pure utility & domain functions
│   └── style.css                # Responsive design with CSS custom properties
├── Case Study 1/                # Technical Workshop Ticket Booking
│   └── index.html               # Form handling, focus management & dual Web Storage
├── Case Study 2/                # Online Music Concert Registration
│   └── index.html               # Ticket registration portal with live event feedback
└── Case Study 3/                # Nagpur City Showcase & Interactive Ticker
```

```
    └── index.html                # CSS marquee ticker, landmark cards & motion
accessibility
```

🔬 Theoretical Background

1. JavaScript Timers & Event Loop

JavaScript execution is single-threaded and driven by an event loop. - **`setInterval(callback, delayMs)`:** Repeatedly executes a callback function with a fixed time delay between each call. - **`setTimeout(callback, delayMs)`:** Schedules a one-time callback after the specified delay. - **`clearInterval(timerId)` / `clearTimeout(timerId)`:** Cancels pending timer executions to avoid memory leaks.

2. ES6 Module Architecture (`import` / `export`)

ES6 modules allow codebases to be partitioned into discrete, reusable files with isolated scopes: - **`export function name(...) { ... }`:** Exposes specified functions or variables from a module. - **`import { name } from './module.js'`:** Ingests selected definitions into the consuming script. - **`<script type="module" src="app.js">`:** Instructs the browser to treat the script as an ES module, enabling strict mode and deferred execution by default.

💻 Section 1: Core Practical — Real-Time Class Scheduler

Modular Architecture

The scheduler is cleanly decomposed into a domain logic module (`scheduler.js`) and a user interface controller (`app.js`).

Utility Module (`scheduler.js`)

Contains pure functions for date arithmetic, formatting, and collision evaluation: - **`classTime(date, time)`:** Combines date keys and military time strings into JavaScript `Date` instances. - **`formatCountdown(ms)`:** Converts millisecond deltas into formatted `Xh Ym Zs` human-readable countdown strings. - **`statusOf(item, now)`:** Evaluates current time relative to class bounds to determine current status: - If `now >= end` ➔ **"Completed"** - If `now >= start` ➔ **"In progress"** - Otherwise ➔ **"Starts in"** - **`overlaps(classes, candidate)`:** Validates against schedule conflicts using interval comparison: `javascript export function overlaps(classes, candidate) { return classes.some((item) => item.date === candidate.date && candidate.start < item.end && candidate.end > item.start); }`

Application Controller (`app.js`)

- Runs a 1000ms clock heartbeat via `setInterval(render, 1000)` to refresh the active digital clock and countdown timers in real time.
- Manages local schedule storage and day-by-day navigation ("Previous Day", "Today", "Next Day").
- Handles class submission, executes collision validation, and displays contextual feedback alerts.

🏢 Section 2: Case Studies

Case Study 1: Technical Workshop Ticket Booking

- **Event Lifecycle Listeners:**
- `focusin` & `focusout`: Dynamically updates border accent hues and background highlights on active inputs.
- `change`: Monitors workshop tracks and student year selections.
- `submit`: Prevents default form reload, extracts inputs via `Object.fromEntries(new FormData(form))`, and synchronizes data to both `localStorage` and `sessionStorage`.

Case Study 2: Online Music Concert Registration

- **Interactive Event Registration:** Provides intuitive ticket tier selectors and participant validation.
- **Real-Time Visual Validation:** Instant visual feedback when ticket allowances or required fields meet validation constraints.

Case Study 3: Nagpur City Showcase & Animated Marquee Ticker

- **Dynamic CSS Ticker Marquee:** Utilizes `@keyframes scroll-message` to deliver a smooth information crawl across the footer.
- **User Control & Accessibility:** Suspends animation on user hover or keyboard focus (`animation-play-state: paused`) and complies with `@media (prefers-reduced-motion: reduce)`.

🎯 Learning Outcomes

- Acquired mastery over JavaScript timers (`setInterval`, `setTimeout`) and real-time state synchronization.
- Understood ES6 module bundling and separation of concerns (`scheduler.js` vs `app.js`).
- Implemented robust collision detection algorithms and multi-state countdown timers.
- Integrated modern form event handling patterns with resilient client storage architectures.